

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse realworld problem.		
Student Name:	RAGUL M	Reg. No.:	192524066
Section / Batch:	SLOT-A CSA0305	Date:	04/09/2026

Code of Conduct

I RAGUL/192524066 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ragul M

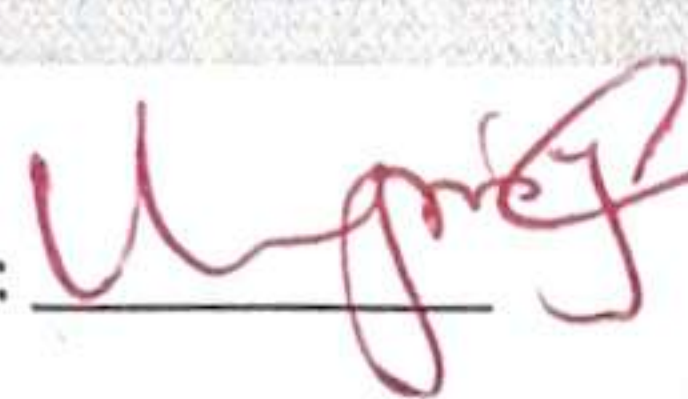
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20

Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

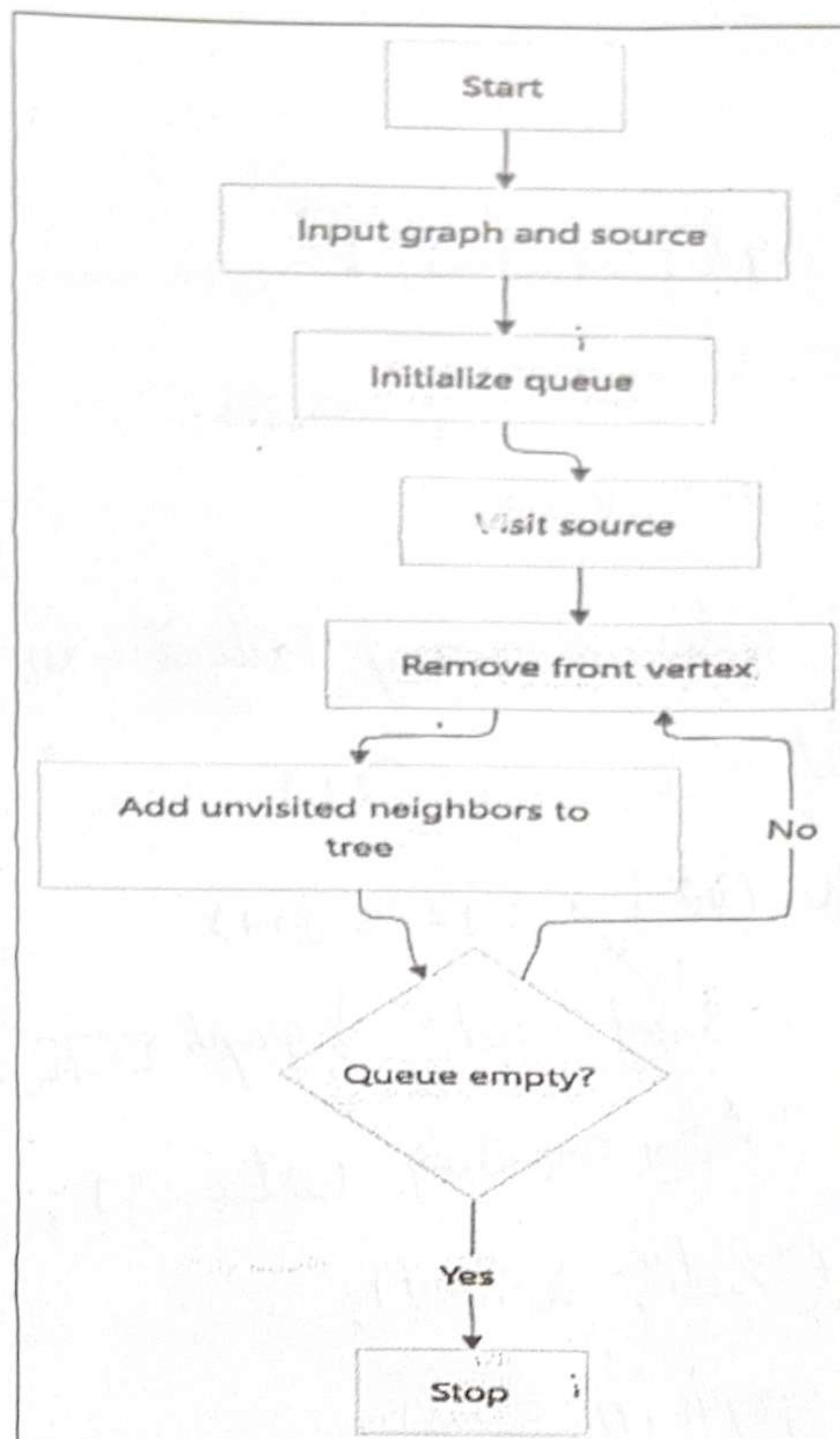
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or nonfunctional code
Code Quality & Readability	20	Clean, modular, wellcommented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence
Faculty In-charge		Course Coordinator		Program Director	
Signature: 		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

Flowchart Diagram:



```

1. #include <stdio.h>
# define max 100
int queue [max], front=0, rear = 0;
int visited [max];
void BFS (int graph [max][max], int n, int start)

```

```

{
    queue [rear++] = start;
    visited [start] = 1;
    while (front < rear)
    {
        int u = queue [front++];
        printf ("%d ", u);
        for (int v=0; v<n; v++)
        {
            if (graph [u][v] == 1 && !visited [v])

```



```

    }
    visited[v] = 1;
    queue[rear++] = v;
}
}
}
}

```

```

int main()

```

```

{
    int n, graph[MAX][MAX], start;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency Matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d", &start);

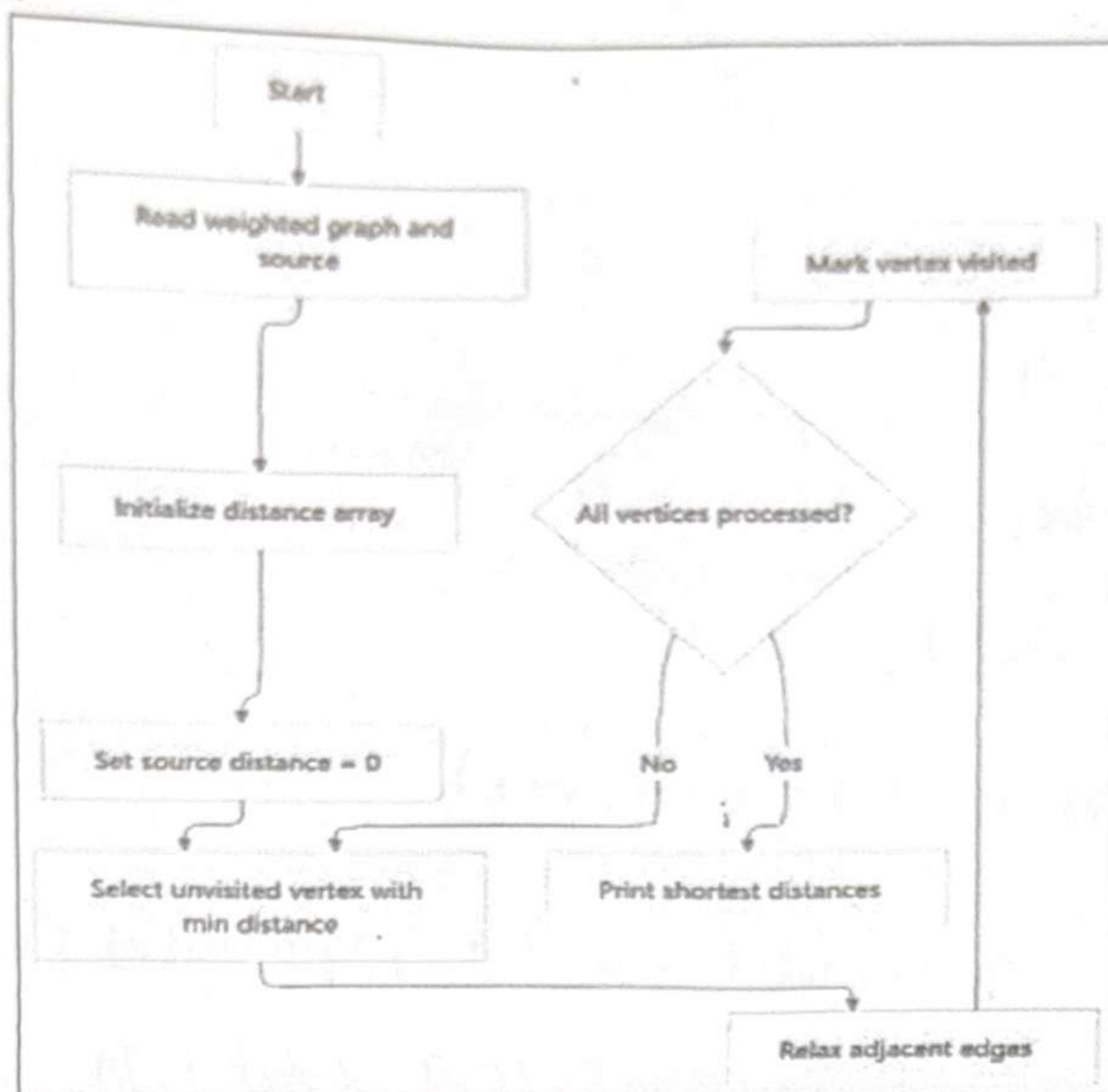
    BFS(graph, n, start);

    return 0;
}

```


2. Convert the flowchart for Dijkstra's shortest path algorithm into code.
Flowchart Diagram:

(20 Marks)



CODE

```
#include <stdio.h>
```

```
#define MAX 100
```

```
#define INF 99999
```

```
void dijkstra (int graph [MAX][MAX], int n, int start)
```

```
{
    int dist [MAX], visited [MAX];
```

```
    for (int i = 0; i < n; i++)
```

```
    {
        dist [i] = INF;
```

```
        visited [i] = 0;
```

```
    }
```

```
    dist [start] = 0;
```

```
    for (int count = 0; count < n - 1; count++)
```

```
    {
        int min = INF, u = -1;
```

```
        for (int i = 0; i < n; i++)
```



```
{ if (!visited[i] && dist[i] < min)
```

```
{ min = dist[i];
```

```
  v = i;
```

```
}
```

```
}
```

```
if (v == -1)
```

```
  break;
```

```
visited[v] = 1;
```

```
for (int v = 0; v < n; v++)
```

```
{ if (visited[v] && graph[v][v] != 0 &&
```

```
  dist[v] + graph[v][v] < dist[v])
```

```
{ dist[v] = dist[v] + graph[v][v];
```

```
}
```

```
printf("Shortest distance: \n");
```

```
for (int i = 0; i < n; i++)
```

```
printf("%d -> %d = %d \n", start, i, dist[i]);
```

```
}
```

```
int main()
```

```
{
```

```
  int n, graph[Max][Max], start;
```

```
  scanf("%d", &n);
```

```
  for (int i = 0; i < n; i++)
```

```
    for (int j = 0; j < n; j++)
```

```
      scanf("%d", &graph[i][j]);
```

```
  scanf("%d", &start);
```

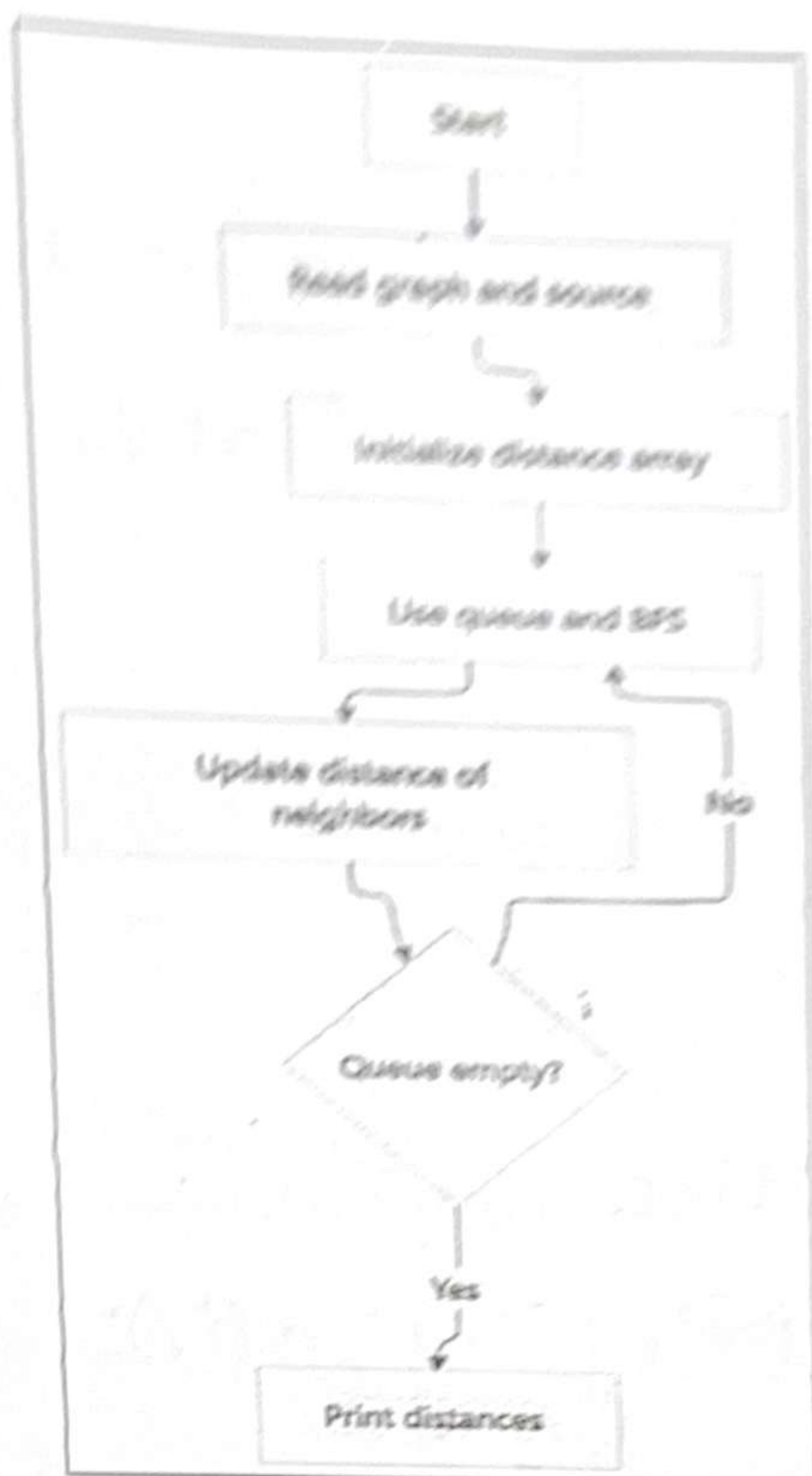
```
  dijkstra(graph, n, start);
```

```
  return;
```

```
}
```


3. Convert the flowchart for shortest path in an unweighted graph into code.
Flowchart Diagram:

(20 Marks)



CODE:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int g[MAX][MAX], q[MAX], d[MAX];
```

```
    int n, s, f = 0, m = 0;
```

```
    scanf("%d %d", &n, &s);
```

```
    for (int i = 0; i < n; i++)
```

```
    { d[i] = -1;
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &g[i][j]);
```

}

$q[u] = 0$

$q[u] = 0$

while $r \neq 0$

and $u = q[u]$

for $v = 0; v < n; v++$

$\{ \text{if } (u[v] \neq 0) \text{ then } u[v] = 1 \}$

$\{ \text{if } (u[v] = 0) \text{ then } u[v] = 1 \}$

$\{ \text{if } (u[v] = 1) \text{ then } u[v] = 0 \}$

$\{ \text{if } (u[v] = 0) \text{ then } u[v] = 1 \}$

for $v = 0; v < n; v++$

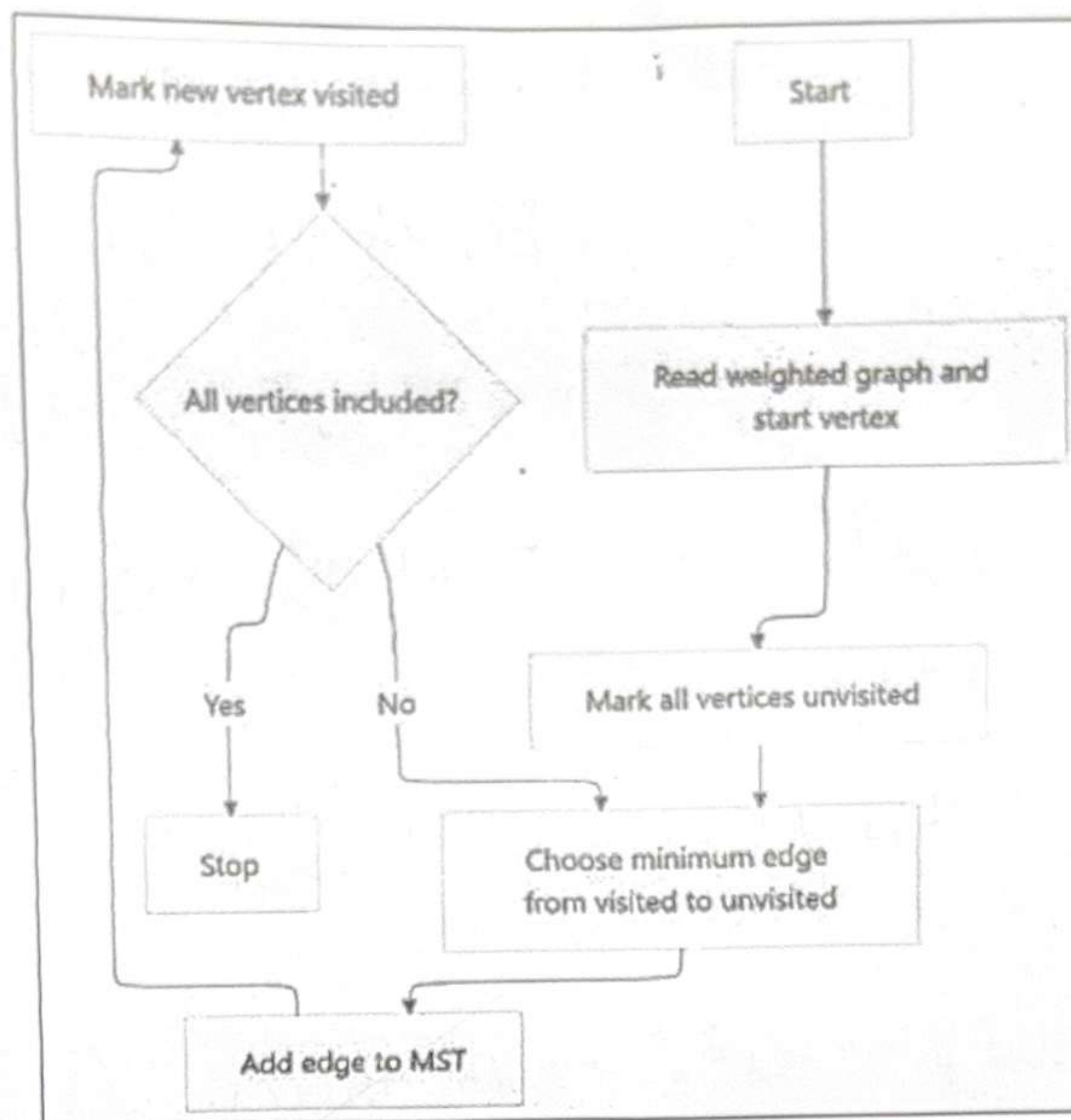
return (u)

$\{ \}$

4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



CODE:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 9999
```

```
int main() {
```

```
    int g[MAX][MAX], s[MAX] = {0};
```

```
    int n, min, x, y, total = 0;
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &g[i][j]);
```

```
            s[i] = 1;
```

```
            for (int e = 0; e < n - 1; e++)
```

```
                {
```

```
                    min = INF;
```



```
for(int i=0; i<n-1;
```

```
for(int j=0; j<n; j++)
```

```
if (S[i]
```

```
for(int j=0; j<n; j++)
```

```
if (!S[j] && g[i][j] && g[i][j] < min)
```

```
min = g[i][j], x=i, y=j;
```

```
printf("%d %d\n", x, y, min);
```

```
total += min;
```

```
S[y]=1;
```

```
}
```

```
printf("cost = %d", total);
```

```
}
```


5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:



CODE:

```
#include <stdio.h>
```

```
struct edge {
```

```
    int u, v, w;
```

```
};
```

```
int parent[10];
```

```
int find(int x) {
```

```
    return parent[x] == x ? x : find(parent[x]);
```

```
}
```

```
int main()
```

```
{    struct edge e[20], t;
```

```
    int n, m, count = 0, total = 0;
```

```
    scanf("%d %d", &n, &m);
```

```
    for(int i = 0; i < m; i++)
```

```
        scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```



```
for (int i=0; i<n; i++) parent[i] = i;
```

```
for (int i=0; i<m-1; i++)
```

```
for (int j=0; j<m-i-1; j++)
```

```
if (e[j].w > e[j+1].w)
```

```
t = e[j], e[j] = e[j+1], e[j+1] = t;
```

```
for (int i=0; i<m && count < n-1; i++)
```

```
{ int a = find(e[i].u);
```

```
int b = find(e[i].v);
```

```
if (a != b)
```

```
{ printf("%d - %d = %d\n", e[i].u, e[i].v, e[i].w);
```

```
total += e[i].w;
```

```
parent[b] = a;
```

```
count++;
```

```
}
```

```
printf("cost = %d", total);
```

Wrong